amU Faculté des sciences Aix Marseille Université

Année universitaire 2024/2025

Site : ☐ Luminy ☒ St-Charles ☐ St-Jérôme ☐ Cht-Gombert ☐ Aix-Montperrin ☐ Aubagne-SATIS
Sujet de : ☐ 1er semestre ☒ 2ème semestre ☐ Session 2 Durée de l'épreuve : 2h
Examen de : L2 Nom du diplôme : Licence Informatique parcours Informatique
Code du module : SIN4U27 Libellé du module : Programmation C
Calculatrices autorisées : NON Documents autorisés : NON

Nom : ____________________
Prénom : ____________________
N° étudiant : ____________________

Répondre directement sur l'énoncé. Garder l'énoncé agrafé.

1 Manipulation de chaînes

On rappelle la fonction `strncmp(char const * s1, char const * s2, size_t n)`, qui permet de comparer deux chaînes selon leurs `n` premiers caractères.

Exercice 1.1. *Quelle directive autorise l'utilisation de cette fonction dans le programme ?*

```
#include ____________________
```

Exercice 1.2. *Écrire une fonction permettant de calculer la longueur du plus long préfixe* **strict** *d'une chaîne qui en soit aussi un suffixe. Par exemple,*

- *pour `"entente"` la valeur renvoyée est 4 (puisque `"ente"` est à la fois préfixe et suffixe) ;*
- *pour `"salsa"`, la valeur renvoyée est 2 (avec `"sa"`) ;*
- *s'il n'y a pas de préfixe strict qui soit aussi un suffixe, comme dans `"programme"`, la valeur retournée est 0.*

```
size_t MyString_longestPrefixSuffix(char const * source) {
  size_t slen = ____________________;
  for (__________; __________; __________) {
    if (strncmp(__________, __________, __________) == 0)
      return __________;
  }
  return 0;
}
```

Exercice 1.3. *Écrire une fonction* `strnchr`*, qui renvoie l'adresse du premier caractère égal au caractère* `c` *dans le préfixe de longueur au plus* `n` *d'une chaîne* `string`*, ou* `NULL` *si le caractère* `c` *n'apparaît pas. Par exemple,*

- *`strnchar("programme",4,'g')` renvoie l'adresse d'une chaîne `"gramme"` ;*
- *`strnchar("programme",4,'r')` renvoie `NULL`, car aucun `'r'` n'apparaît dans les 4 premières lettres du mot.*

```
char * MyString_strnchr(char const * string, size_t n, int c) {
  char ch = c;
  for (size_t index = 0; index < n; index++) {
    if (                              ) return                        ;
    if (                              ) break;
  }
  return NULL;
}
```

Exercice 1.4. *Écrire une fonction renvoyant une chaîne contenant tous les caractères apparaissant au moins une fois dans la chaîne* `source`*. Chaque caractère présent dans* `source` *doit être présent exactement une fois dans la chaîne produite, par exemple,* `"ananas"` *donnera* `"ans"`*. La chaîne retournée doit être allouée avec un bloc de taille minimum (on procédera à une réallocation du bloc), et doit être terminée par un* `'\0'`*. Utiliser* `strnchar` *pour déterminer si un caractère a déjà été listé.*

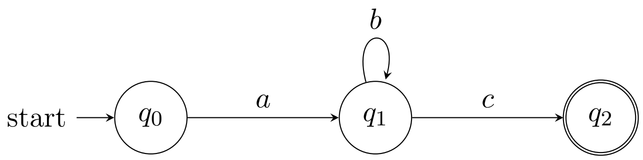
```
char * MyString_charSet(char const * source) {
  size_t len = strlen(source);
  char * set =                                              ;
  if (                                      ) return NULL;
  size_t setCount = 0;
  for (size_t i = 0; i < len; i++) {
    if (                                      ) continue;
                             = source[i];
                             ;
  }
                                = '\0';
  return realloc(               ,                               );
}
```

2 Implémentation d'un automate

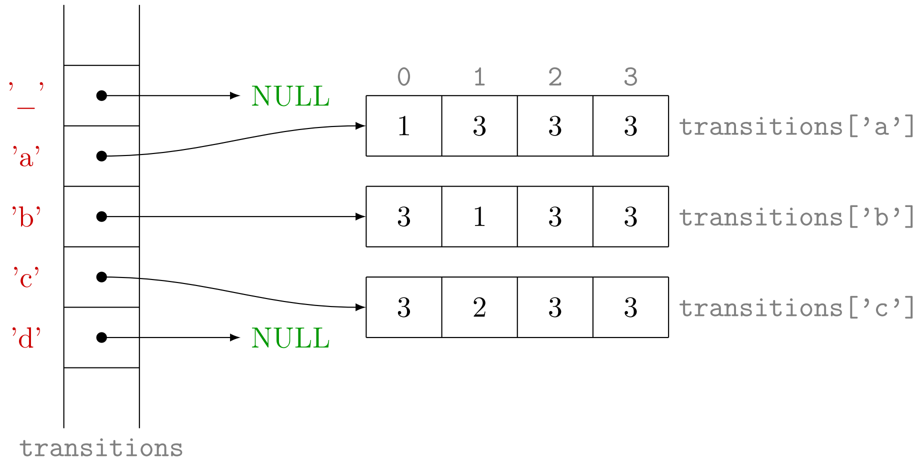
On souhaite implémenter des automates à états finis déterministes, sur l'alphabet des caractères non-signés. Les états d'un automate seront encodés comme les entiers de 0 à `stateCount-1`. On dispose aussi

- de l'état initial `initialState` ;
- d'un état par défaut ou état d'échec `defaultState` ;
- d'un tableau de booléens `acceptingStates`, de sorte que `acceptingStates[2]` vaut `true` si l'état 2 est acceptant ;
- de la fonction de transition `transitions` représentée par un tableau bi-dimensionnel. Ainsi, l'état obtenu en partant de l'état 3 à la lecture du caractère `'a'` est représenté par `transition['a'][3]`.

Voici un exemple d'automate, pour le langage ab^*c, auquel il faudrait ajouter un *état d'échec* ou *état par défaut* q_3 vers lequel pointe toutes les transitions non-dessinées.



De plus, pour économiser la mémoire, les transitions vers l'état d'échec ne sont pas forcément représentées en mémoire. Précisément, les seules lettres `c` ayant un tableau `transitions[c]` alloué, sont celles étiquetant au moins une transition vers un état autre que celui d'échec. Si `transitions[c] == NULL` alors la lecture de la lettre `c` mène à l'état d'échec. Le tableau des transitions de cet automate correspond donc au schéma :



Pour résumé, voici la définition d'un automate.

```
#define CHAR_COUNT (1 + (size_t) UCHAR_MAX)

typedef struct {
  int stateCount;
  int initialState;
  int defaultState;
  bool * acceptingStates;
  int * transitions[CHAR_COUNT];
} Automaton;
```

Exercice 2.1. *Compléter les deux fonctions permettant de modifier l'état par défaut et l'état initial d'un automate existant* `self`*.*

```
void Automaton_setDefaultState(Automaton * self, int defaultState) {
                                                                          ;
}

void Automaton_setInitialState(Automaton * self, int initialState) {
                                                                          ;
}
```

Exercice 2.2. *Écrire une fonction déterminant si un état* `state` *d'un automate* `self` *est acceptant.*

```
bool Automaton_isAccepting(Automaton * self, int state) {
  return ____________________;
}
```

Exercice 2.3. *Écrire une fonction qui modifie l'automate `self`, en rendant l'état `state` acceptant si `accepts` est vrai, non-acceptant sinon.*

```
void Automaton_setStateAcceptance(Automaton * self, int state, bool accepts) {
  ____________________ = accepts;
}
```

Exercice 2.4. *Écrire une fonction permettant de calculer l'état `nextState` de l'automate `self` suite à la lecture d'un caractère `c` depuis l'état `currentState`.*

```
int Automaton_nextState(Automaton * self, char c, int currentState) {
  unsigned char uc = c;
  size_t cIndex = uc;
  int * cStates = ____________________;
  if (__________) return __________;
  return ____________________;
}
```

Exercice 2.5. *Dans les fonctions précédentes, il aurait été désirable d'ajouter une assertion pour vérifier que l'état `state` (ou `currentState`) reçu en paramètre est bien valide pour l'automate `self`. Quelle est cette assertion ?*

```
assert(____________________);
```

Exercice 2.6. *Compléter la fonction suivante, décidant si un mot `word` est reconnu par l'automate `self`.*

```
bool Automaton_acceptsWord(Automaton * self, char const * word) {
  size_t len = strlen(word);
  int currentState = ____________________;
  for (__________; __________; __________) {
    currentState = ____________________;
  }
  return ____________________;
}
```

Exercice 2.7. *Écrire la fonction d'initialisation d'un automate, avec un nombre d'états `stateCount` donné. Initialement, l'état initial et l'état d'échec sont mis à 0, aucun n'état n'est acceptant et toutes les transitions mènent à l'état d'échec. Il faut initialiser les tableaux `acceptingStates` (à `false`) et `transitions` (à `NULL`).*

```
void Automaton_init(Automaton * self, int stateCount) {
  self->stateCount = stateCount;
  self->defaultState = 0;
  self->initialState = 0;
  self->                    = malloc(                                             );
  assert(                                                  );
  for (int i = 0; i <                              ; i++) {
    self->                                        = false;
  }
  for (int c = 0; c <                              ; c++)  {
    self->                                        = NULL;
  }
}
```

Exercice 2.8. *Ajouter une fonction, qui pour un caractère* `c`*, s'assure que* `transitions[c]` *n'est pas* `NULL`*. S'il est* `NULL`*, y allouer un nouveau tableau qui contiendra la fonction de transition pour ce caractère* `c`*, initialisé à l'état d'échec.*

```
void Automaton_ensureLetter(Automaton * self, unsigned char c) {
  if (                                        ) return;
                                = malloc(                                             );
  assert(                                                  );
  for (int i = 0; i <                              ; i++) {
                                                 = self->defaultState;
  }
}
```

Exercice 2.9. *En utilisant la fonction de l'exercice précédent, compléter la fonction suivante, qui ajoute (ou remplace) la transition depuis l'état* `fromState` *et la lettre* `c`*, pour l'envoyer vers l'état* `toState`*.*

```
void Automaton_setTransitionFromTo(Automaton * self, char c, int fromState, int toState) {
  unsigned char uc = (unsigned char) c;
  size_t cIndex = (size_t) uc;
                                                                            ;
                                                                            ;
}
```